

Dirty Spokes Dirtbike Rental Management System

Step 4 Draft

Web App URL: <http://classwork.engr.oregonstate.edu:41801/>

a) Project Outline, DB Outline, ERD, Schema Diagram, & Sample Data

Overview

Dirty Spokes is a recreational dirtbike rental facility and track located in sunny Southern California, operating a fleet of 50 bikes across 8 major brands including Honda, Yamaha, and Kawasaki. The facility services roughly 500 unique customers per year across an estimated 2,500 annual rental sessions. Currently, staff manage bike inventory, customer records, and rental history through manual bookkeeping and paper waivers.

Over the past year, this has led to a number of operational issues including customer records being lost or duplicated, no reliable way to track when a customer had which bike at any given time, and difficulty identifying when a bike was last rented or how many hours it has accumulated.

A database-driven web application will allow Dirty Spokes administrators to manage Bikes, Brands, Customers, and Rentals in a unified system, ensuring accurate rental records, up-to-date usage metrics for bike maintenance and popularity, and smoother day-to-day operations at the track.

In applying the normalization process to this design, no changes to the outline or ERD are required after Step 1's feedback fixes. The schema is in 3NF. All attributes contain atomic values with no repeating groups, all non-key attributes are fully dependent on their primary keys with no partial dependencies, and no non-key attribute determines another non-key attribute.

Foreign key ON DELETE and ON UPDATE behaviors have been documented in Step 3 for clarity on relationships in the database outline.

The DDL has been wrapped in a stored procedure, `reset_db()`. This Step 4 addition is invoked by a RESET button on each entity page. Rather than implementing a single demo CUD procedure, full CRUD was implemented for all four entities. Each entity has dedicated `sp_Add`, `sp_Update`, and `sp_Delete` stored procedures wired to POST routes in `app.js`. Any and every Add/Edit/Delete form or action can be used to demo CUD and RESET functionality.

Database Outline

Bikes: records the dirtbikes available in the rental fleet

- bikeID: INT UNSIGNED, AUTO_INCREMENT, unique, NOT NULL, PK
- frameNumber: VARCHAR(50), unique, NOT NULL
- brandID: INT UNSIGNED, NOT NULL, FK → Brands.brandID
- modelName: VARCHAR(100), NOT NULL
- engineSize: VARCHAR(10), NOT NULL - e.g. "250cc", "450cc"
- bikeYear: INT UNSIGNED, NOT NULL
- engineHourMeter: DECIMAL(6,1), NOT NULL
- Relationships:
 - M:1 between Bikes and Brands, many bikes can belong to one brand implemented by FK brandID. ON DELETE RESTRICT prevents a brand from being deleted while bikes are associated with it. ON UPDATE CASCADE propagates brand ID changes to all associated bikes.
 - 1:M between Bikes and Rentals, one bike can appear in many rental records implemented by FK bikeID in Rentals
 - M:N between Bikes and Customers, a bike can be rented by many customers and a customer can rent many bikes, implemented through the Rentals intersection table

Brands: records the manufacturers of the bikes in the fleet

- brandID: INT UNSIGNED, AUTO_INCREMENT, unique, NOT NULL, PK
- brandName: VARCHAR(100), unique, NOT NULL
- countryOfOrigin: VARCHAR(100), NOT NULL
- localDealer: VARCHAR(255), NOT NULL
- Relationships:
 - 1:M between Brands and Bikes, one brand can have many bikes implemented by brandID FK in Bikes

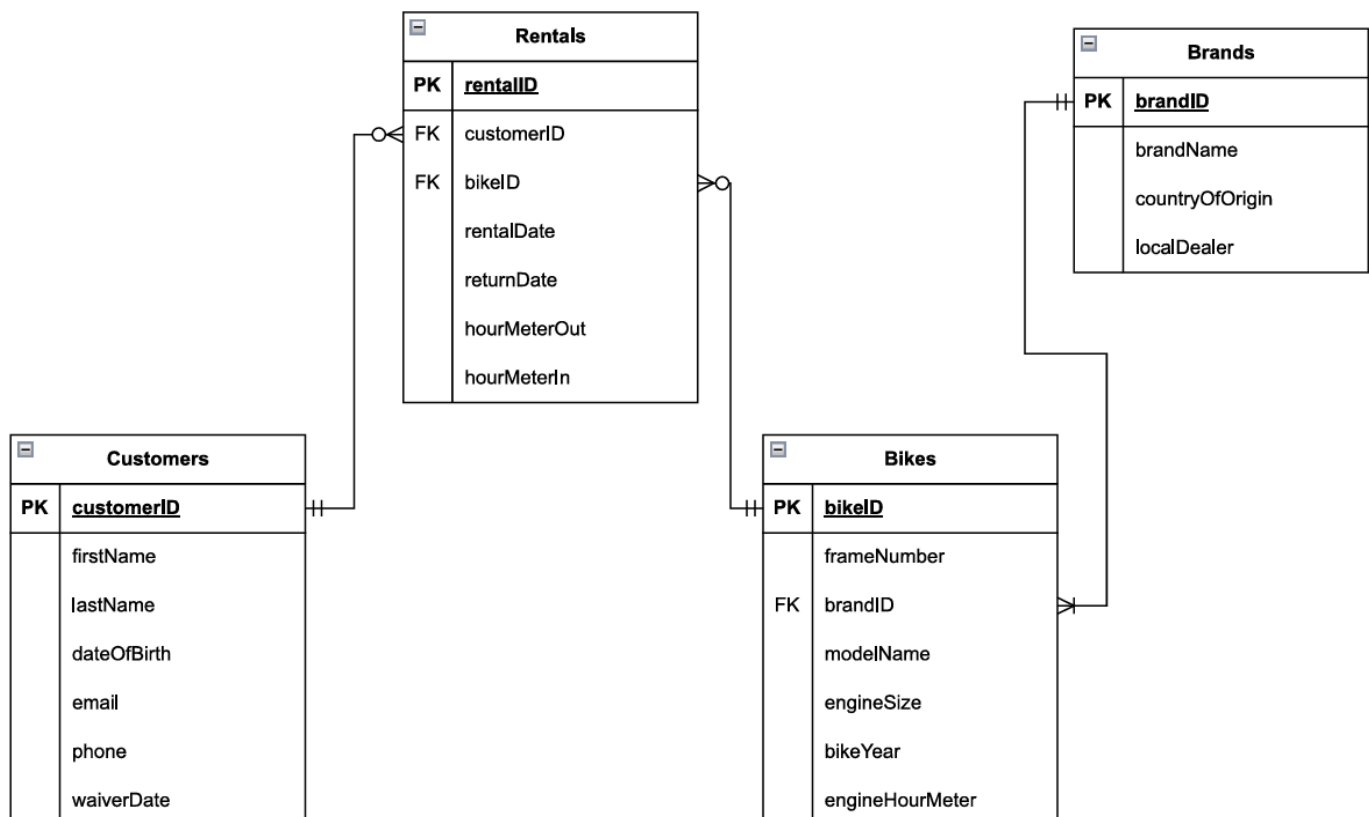
Customers: records customers who rent bikes at the track

- customerID: INT UNSIGNED, AUTO_INCREMENT, unique, NOT NULL, PK
- firstName: VARCHAR(50), NOT NULL
- lastName: VARCHAR(50), NOT NULL
- dateOfBirth: date, NOT NULL
- email: VARCHAR(255), unique, NOT NULL
- phone: VARCHAR(20), NOT NULL
- waiverDate: date, NOT NULL
- Relationships:
 - 1:M between Customers and Rentals, one customer can have many rental records implemented by FK customerID in Rentals
 - M:N between Customers and Bikes, a customer can rent many bikes and a bike can be rented by many customers, implemented through the Rentals intersection table

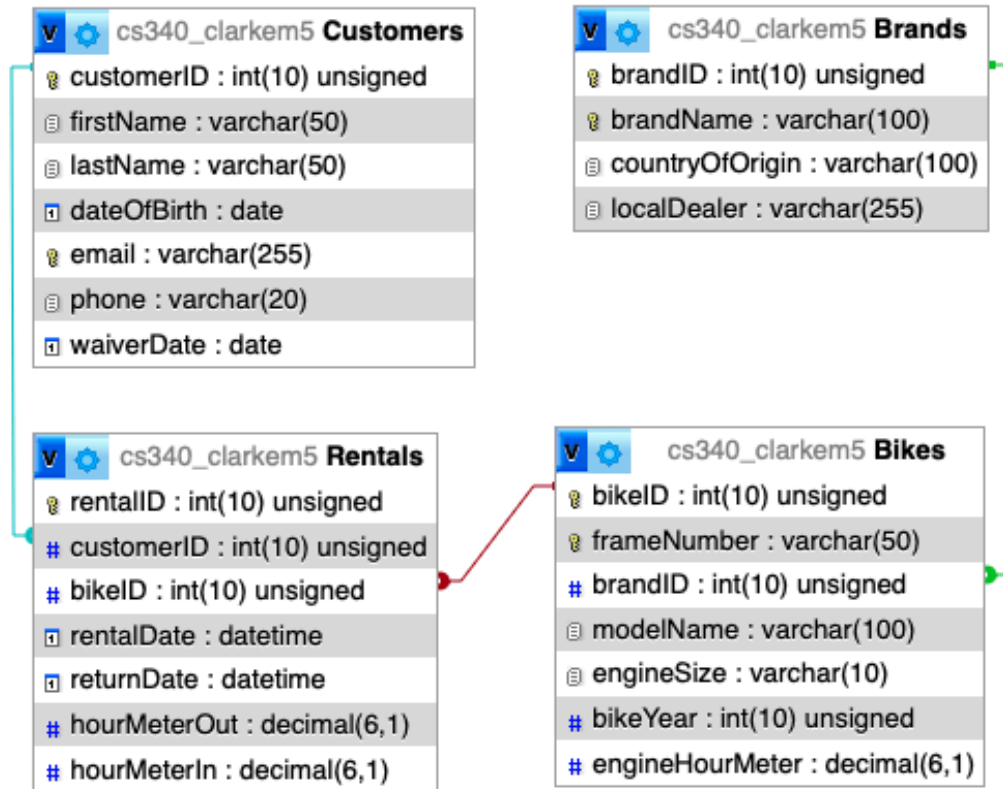
Rentals: intersection table recording which customer rented which bike and when

- rentalID: INT UNSIGNED, AUTO_INCREMENT, unique, NOT NULL, PK
- customerID: INT UNSIGNED, NOT NULL, FK → Customers.customerID
- bikeID: INT UNSIGNED, NOT NULL, FK → Bikes.bikeID
- rentalDate: datetime, NOT NULL
- returnDate: datetime, NOT NULL
- hourMeterOut: DECIMAL(6,1), NOT NULL
- hourMeterIn: DECIMAL(6,1), NOT NULL
- Relationships:
 - M:1 between Rentals and Customers, many rental records can belong to one customer where Customers.customerID is PK and Rentals.customerID is FK. ON DELETE CASCADE automatically removes Rental records associated with a Customer when that Customer is deleted. ON UPDATE CASCADE propagates ID changes.
 - M:1 between Rentals and Bikes, many rental records can reference one bike where Bikes.bikeID is PK and Rentals.bikeID is FK. ON DELETE CASCADE automatically removes Rental records associated with a Bike when that Bike is deleted. ON UPDATE CASCADE propagates ID changes.
 - M:N between Customers and Bikes, a customer can rent many bikes and a bike can be rented by many customers

Entity-Relationship Diagram



Schema Diagram



Example Data

Bikes Table

bikeID	frameNumber	brandID	modelName	engineSize	bikeYear	engineHourMeter
1	JYACB11C0RA015607	3	YZ65	65cc	2024	15.8
2	VBKXWM236MM321033	2	300 XC-W	293cc	2021	71.4
3	VNBS648C4PB000307	1	SE-F 500 Factory 4T	478cc	2023	225.5

Brands Table

brandID	brandName	countryOfOrigin	localDealer
1	Sherco	France	Trials Offroad
2	KTM	Austria	SoCal Moto
3	Yamaha	Japan	SoCal Moto

Customers Table

customerID	firstName	lastName	dateOfBirth	email	phone	waiverDate
1	Jeremy	Stevens	1989-06-11	jsteve0@email.com	555-0101	2024-11-22
2	Andrea	Hernandez	2003-11-25	andrea44@email.com	555-0202	2025-04-17
3	Roman	Lutoslawski	1978-08-26	roman444@email.com	555-0303	2020-08-20

Rentals Table

rentalID	customerID	bikeID	rentalDate	returnDate	hourMeterOut	hourMeterIn
1	3	3	2025-04-06 11:22:56	2025-04-08 16:43:16	219.9	225.5
2	3	2	2026-04-10 09:30:17	2026-04-13 10:17:44	60.6	68.0
3	2	2	2026-04-19 10:19:19	2026-04-19 15:22:04	68.1	71.4

b) Fixes based on Feedback from Previous Steps:

Step 1 Feedback Verbatim

Review by Michael Santora:

Hi Spencer and Matthew!

This is really impressive work on the overview. You painted an excellent picture of the business model while managing to highlight the key operational issues a DBMS could help solve. The objective in paragraph three is clearly stated: creating unification while ensuring data accuracy and real-time tracking. Finally, the included image was also a nice touch to make the scenario feel more "real!"

The four entities you've identified are logical, and it was a smart design choice to make each table's ID field specific (e.g., customerID and rentalID) rather than using a generic "ID" across the board. To further refine your schema, I suggest the following:

- For data types like VARCHAR or INT, consider specifying the maximum length or display width.
- It may be worth noting which INT attributes should be signed vs. unsigned.
- Perhaps another table to house Employee would be beneficial for this DBMS.
- I see you've noted the Rentals table as an intersection table to facilitate the M:N relationship between Bikes and Customers. However, since it houses specific event data, it also could function as a transaction table. It may be worth pivoting to make Rentals a standard transaction table and create a dedicated intersection table (e.g., CustomersBikes) containing only the Foreign Keys. Regardless, I believe your current setup to be quite functional for this.

I really like your ERD model because it shows perfect consistency with your written overview. You correctly followed the standard naming conventions by keeping entities plural and attributes singular (e.g., Customers, customerID). The tables and respective attributes match the outline perfectly, and the Crow's Foot notation makes the logic very easy to follow!

Lastly, thank you for clarifying the sources you leveraged for this stage of the project.

Nice work, both!

No external sources were leveraged in my review at this time.

Review by Joshua York:

I agree, Dirty Spokes, dirt bike rentals could use a database and web site to track the rentals, customers, and bikes from the overview. 50 bikes over 2500 annual rentals could be difficult to manage with paper receipts. Yes, the overview lists specific facts that support the use of the DB and website. Specifically, the overview goes over lost records or duplicate records, a lack of accurately tracking rental hours, and which customer had which bike using their current method.

There are 4 entities described and each represent a single idea. Bikes, Brands, Customers, and Rentals are all separate ideas. The Rentals table represents the M:N relationship between Customers and Bikes but it adds additional attributes that make it an entity. The relationship between the entities is described well and there are 1:M and M:N relationships present.

The outline accurately details the purpose and attributes (including datatypes and constraints). The relationships are present and described. Bikes to Brands is described as a Brand can have many Bikes. The relationship between Bikes to Brands I believe should be mandatory instead of optional. As a bike must have a brand.

In the ERD, there's a connection from Customer to Bikes (M:N) that should be removed. It's not referenced in the database outline, and that relationship is covered by the Rentals table. Cardinality is missing in the same way as the outline describes the relationships. I'm not certain, but these look like mandatory lines instead of optional lines.

There is a consistent naming convention. The table names are plural as required (Bikes, Customers, Rentals, Brands). Attribute names are correctly singular, (engineSize, email, phone, hourMeterOut). Group 25, Matthew and Spencer, good job!

Thanks for posting.

This review was done without any AI assistance. All my work.

(moved this from a comment to an answer)

Review by Devon Carter:

Hi Team!

Starting off, I love your overview. It clearly explains an issue, illustrates the scale with precise figures of bikes, brands, customers and rental sessions, and explains how your database will effectively solve the stated problem.

You have the required four entities, each with clearly explained scope and purpose, and matching the figures stated in your overview. Further, your entity details are consistent with what one would expect recorded for this type of business, and your relationships and foreign keys are very clearly explained throughout.

One change I may make would be to replace the varchar in the phone attribute within Customers with int, removing the standard (XXX)XXX-XXXX format, but preventing possible typographic errors which place non-numerical characters in the entry.

Looking through the ERD, it appears consistent with the information presented in your database outline, and neatly organized to be clearly readable. I would suggest removing the M:N connector between Bikes and Customers, as that is expressed through their intersection in Rentals.

1:M relationships are properly expressed through Foreign Keys, and there is an M:N relationship properly expressed using Rentals as an intersection.

Naming of entities is mostly consistent: Entity names are plural and capitalized, and attributes are camel case and mostly singular, with the sole exception of the engineHours attribute under Bikes which should likely be engineHour for consistency.

All work above is original and written by myself.

Review by Jonathan Marchand:

Does the overview describe what problem is to be solved by a website with DB back end? If yes, summarize. If not, what changes would better support describing the problem to be solved?

Yes the overview does describe the problem that is being solved. The problem being Dirty Spokes Dirtbike is in need of a system to correctly manage bike inventory, customer records, and rental history. Using a manual paper record system in no way meets the demands of today's business environment. The implementation of a dbms to correct the problems with data redundancy, data accuracy, as well as the location of valuable inventory definitely seems like a requirement for a rental business where the inventory is the main generator of revenue. I think this is a great idea and you stated your solution very clearly.

Does the overview list specific facts? If yes, summarize what the facts illustrate about the proposed DB solution. If not, what facts would better support illustrating the scope and scale of the proposed DB solution?

Yes the overview lists specific facts specifically the number of customers, rental sessions, bikes in their rental fleet, as well as the number of rental sessions per year. I think what you have so far is sufficient enough to build out a capable dbms for this fleet management issue.

One suggestion would be to incorporate a staff table as well. Data integrity is invaluable and controlling who has access to what features can be an intelligent implementation regarding data security and tracking updates to the dbms. This concept may go beyond this class but the concept of least privilege is very important in cybersecurity/data security.

Are at least four entities described, and does each one represent a single idea to be stored as a list? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

Yes there are at least 4 entities; Rentals, Brands, Customers, and Bikes.

The many to many relationship featured in their ER diagram is featured by the Rentals table

Many customers can rent many bikes

A Brand can have many Bikes (one to many).

(i.e. bike models per brand, engine sizes),

Each Bike can have many Rentals (one to many).

This is sufficient and meets the requirements for the project. I have no suggestions to improve at this time.

Does the outline of entity details describe the purpose of each, list attribute datatypes and constraints, and describe relationships between entities? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

Yes the outline of the describes the purpose of each well. It is fairly straightforward the purpose of each table and the relationship with each other.

Some suggestions:

For the Brands table you could change localDealer to just dealer or remove the NOT NULL. Brands go out of business and dealers may not carry the product anymore even if it is in your inventory or for sale on the used market. Keeping the NOT NULL may be a little too strict. Just a suggestion.

For customers:

You could have a row for license and set that to a BOOL, or even insurance and set that to BOOL

Are 1:M relationships correctly formulated? Is there at least one M:M relationship? Does the ERD present a logical view of the database? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

Yes the 1:M relationships are formulated correctly. Yes there is one M:M relationship (man Customers can rent many bikes and connects through a join table Rentals). The rentals table shows the correct foreign keys (customerID from Customers and bikeID from Bikes).

Is there consistency in a) naming between overview and entity/attributes b) entities plural, attributes singular c) use of capitalization for naming? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

Yes, the naming convention is very consistent. I don't have any suggestions for improvement at this time.

Review by TA Razeel Hassan Ibrahim Rasheed:

Not sure why did you connect Bikes with customer directly while also stating rentals is intersection table. Intersection table serve to simplify M:N relationships.

Step 1 Accepted Fixes

- Add lengths to VARCHAR and INT attributes - VARCHAR lengths, INT signs, and DECIMAL placing have been added to all attributes throughout the Database Outline for greater precision and clarity.
- Make Brand mandatory to Bikes - The Brands→Bikes relationship in the ERD has been updated to reflect a mandatory relationship, as every bike in the fleet must be associated with a brand.
- Remove M:N connector between Customers and Bikes in ERD - The direct line between Customers and Bikes has been removed from the ERD. This relationship is already properly represented through the Rentals intersection table and the direct connector was redundant.
- Add cardinality to connections in the ERD - Proper cardinality markers have been added to all relationship lines in the ERD.
- Edit engineHours to singular - engineHours was not present in the Database Outline so it has been added to the Bikes entity and changed to a singular name, engineHourMeter.

Step 1 Declined Suggestions

- Add Employees table - The scope of this system is intentionally limited to bike inventory, customer records, and rental history. This is an administrator-facing database tool, not an employee management system. Adding an Employee table would exceed the intended scope of the project.
- phone attribute changed from VARCHAR to INT - We chose to keep phone as VARCHAR since phone numbers may include formatting characters such as dashes, parentheses, spaces, and country code prefixes. Storing phone as INT would risk data loss and overflow on numbers with country codes as well as prevent storing any non-numeric character formatting.
- Make localDealer optional - This suggestion was not employed as recording the nearest outside facility that sells or services a specific brand is a necessity in assembling a fleet of popular models and maintaining them throughout their lifecycle. The scenario of a major brand ceasing operations and no longer being available is highly unlikely, and the discontinuation of a brand would result in its removal from the active fleet entirely rather than keeping it with a null dealer field.

- Add license and insurance attributes to Customers - This suggestion is not a necessity in this context. 50cc MX bikes and increasingly popular electric bikes of comparable sizes are geared toward riders as young as 5 years old who would not have a license or insurance to record. Beyond this, off-highway vehicles do not require a license or insurance to operate in legal settings such as a designated MX or enduro track. Adding these fields would therefore not be applicable to a significant portion of the customer base and is out of scope for the current system design.

Step 2 Feedback Verbatim

Review by Ikumi Kameyama:

1. Does the schema present a physical model that follows the database outline and the ER logical diagram exactly?

Overall, the schema follows the outline and ERD very closely. The four entities (Bikes, Brands, Customers, and Rentals), match across the outline, ERD, and schema.

A few small observations:

- The ERD includes all attributes now, which matches the outline well.
- The schema diagram is clear, but the ERD uses “bike Year” (with a space) while the schema uses bikeYear. This is a small formatting inconsistency.
- The ERD shows the correct cardinalities, and the schema enforces them with foreign keys.

2. Is there consistency in naming (overview, outline, ER, schema), plural entities, singular attributes, capitalization)?

- Entities use PascalCase (Bikes, Brands, Customers, Rentals).
- Attributes use camelCase or lowercase with no spaces (frameNumber, engineHourMeter, bikeYear).

3. Is the schema easy to read (diagram clear, lines not crossed)?

Yes, the ERD is clean and easy to read. The cardinality markers are clear, and the relationships are not cluttered. The schema diagram is also readable and well-organized.

4. Are intersection tables properly formed (two FKs and facilitate a M:N relationship)?

Yes. The Rentals table correctly acts as the intersection table for the M:N relationship between Customers and Bikes.

- It includes customerID (FK)
- It includes bikeID (FK)
- It includes its own PK (rentalID), which is acceptable
- It includes additional attributes relevant to the rental event

5. Does the sample data suggest any non-normalized issues (partial dependencies or transitive dependencies)?

I did not find any normalization issues.

6. Is the SQL file syntactically correct?

The SQL shown in the schema diagram appears correct.

The example data loads successfully in MariaDB, which suggests the SQL is valid.

7. Are the data types appropriate considering the description of the attribute in the database outline?

- DECIMAL(6,1) for hour meters is correct
- VARCHAR lengths are reasonable
- INT UNSIGNED for IDs is appropriate
- DATETIME for rentalDate and returnDate is correct

8. Are primary and foreign keys correctly defined when compared to the Schema? Are appropriate CASCADE operations declared?

Yes, the primary and foreign keys match the outline and ERD.

The schema enforces the correct relationships.

9. Are relationship tables present when compared to the ERD/Schema?

Yes.

The Rentals table is present and correctly implements the M:N relationship between Customers and Bikes.

10. Is all example data shown in the PDF inserted?

Yes.

The example data in the PDF matches the schema and demonstrates the relationships clearly.

11. Is the SQL well-structured and commented (hand-authored vs. exported)?

The SQL appears hand-authored and well organized.

12. Originality statement

All feedback provided here is my own work based on my understanding of the project. I did not use AI tools to generate this review.

Review by Hannah Kemplin:

Hi Spencer and Matthew!

You all have done some really solid work with your Dirty Spokes rental facility concept. It looks like your project is off to a great start!

- *Does the schema present a physical model that follows the database outline and the ER logical diagram exactly?*

Yes, the schema accurately models the database outline and ERD. The diagram correctly depicts all proposed tables and attributes as well as their respective relationships. The only (very minor) inconsistency I detected was that the bikeYear and engineSize attributes are switched in order on the ER and schema diagrams.

- *Is there consistency in a) naming between overview, outline, ER and schema entity/attributes b) entities plural, attributes singular c) use of capitalization for naming?*

Naming is consistent across overview, outline, ERD, and schema, utilizing plural PascalCase for the entities and singular camelCase for the attributes.

- *Is the schema easy to read (e.g. diagram is clear and readable with relationship lines not crossed)?*

The schema is clear and readable. The layout of the tables is neat, making the relationship lines clean and easy to follow.

- *Are intersection tables properly formed (e.g. two FKs and facilitate a M:N relationship)?*

The Rentals intersection table is properly formed, featuring foreign keys customerID from Customers and bikeID from Bikes and facilitating a M:N relationship between Customers and Bikes.

- *Does the sample data suggest any non-normalized issues, e.g. partial dependencies or transitive dependencies?*

The database design seems to be normalized, achieving 3NF. I could not find any further non-normalized issues.

- *Is the SQL file syntactically correct? This can be easily verified by using phpMyAdmin and your CS 340 database (do not forget to take backup of your own database before you do this!)*

The SQL file was successfully imported into my CS340 database using phpMyAdmin. I did, however, receive an error message indicating a syntax error (screenshot included at the end of my review).

- In the SQL, are the data types appropriate considering the description of the attribute in the database outline?
The data types in the SQL are appropriate given the description of the attribute in the database outline.
- In the SQL, are the primary and foreign keys correctly defined when compared to the Schema? Are appropriate CASCADE operations declared?
Primary keys and foreign keys for all entities are correctly defined, and referential integrity is preserved by the CASCADE operations declared in the creation of the Rentals table.
- In the SQL, are relationship tables present when compared to the ERD/Schema?
All tables, including Bikes, Customers, Rentals, and Brands, are present in the SQL when compared to the ERD and schema.
- In the SQL, is all example data shown in the PDF INSERTED?
Yes, all example data shown in the PDF is properly inserted in the SQL.
- Is the SQL well-structured and commented (e.g. hand authored) or not (e.g. exported from MySQL)?
Overall, the SQL is well-structured. It is commented and appears to be hand authored.
- As a reviewer, clearly describe to what extent your feedback to the team was original (e.g. "all my work") or non-original (e.g. used AI tools per Code Citation Tips).
All my work

✓ Import has been successfully finished, 13 queries executed. (DDL.sql)

Error

Static analysis:

2 errors were found during analysis.

1. Unexpected beginning of statement. (near "Sample" at position 2)
2. Unrecognized statement type. (near "data" at position 9)

SQL query: [Copy](#)

```
--Sample data INSERT INTO Customers( firstName, lastName, dateOfBirth, email, phone, waiverDate ) VALUES ( "Jeremy", "Stevens", '1989-06-11', "jsteve0@email.com", "555-0101", '2024-11-22' ), ( "Andrea", "Hernandez", '2003-11-25', "andrea44@email.com", "555-0202", '2025-04-17' ), ( "Roman", "Lutoslawski", '1978-08-26', "roman444@email.com", "555-0303", '2020-08-20' );
```

MySQL said:

```
#1064 - You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '--Sample data INSERT INTO Customers( firstName, lastName, dateOfB...' at line 1
```

Review by Kayla Haney:

Hi Matthew and Spencer,

This is a well-structured database design with a matching schema and clean SQL. The relationships and entities remain consistent throughout the DDL, sample data and ERD, and I did not find any syntax errors.

Does the schema present a physical model that follows the database outline and the ER logical diagram exactly?

Yes, the schema matches the ERD and database outline.

Is there consistency in a) naming between overview, outline, ER and schema entity/attributes b) entities plural, attributes singular c) use of capitalization for naming?

Yes, there is consistent naming throughout the ER and schema entities/attributes with the appropriate capitalization.

Is the schema easy to read?

The schema is easy to follow with no relationships crossed.

Are intersection tables properly formed (e.g. two FKs and facilitate a M:N relationship)? Does the sample data suggest any non-normalized issues?

The four tables (Customers, Brands, Rentals and Bikes) are clear and have understandable relationships. There is a clear M:N relationship between Customers and Bikes with Rentals as the intersection table for recording which customers rented which bike. The schema is in 3NF with no partial dependencies or transitive dependencies.

The SQL file:

The SQL file is syntactically correct, and the datatypes are appropriate according to the database outline descriptions. The PK and FKs are correctly defined and the CASCADE operations are appropriately declared. These match the schema, ER and database outline.

The sample data is inserted with data that follows the schema. All the example data in the pdf is presented and INSERTED correctly. Overall, the SQL is well-structured with clean, descriptive comments. Great work on the SQL and schema.

This review was all written by me.

Review by Sylvia Nguyen:

Hi Spencer and Matthew!

Does the schema present a physical model that follows the database outline and the ER logical diagram exactly?

Yes the schema presents a physical model that follows the database outline and ERD.

Is there consistency in a) naming between overview, outline, ER and schema entity/attributes b) entities plural, attributes singular c) use of capitalization for naming?

Yes there is consistency in the naming between overview, outline, ERD and schema. The entities are plural and capitalized and the attributes adhere to camelcase.

Is the schema easy to read (e.g. diagram is clear and readable with relationship lines not crossed)?

The schema is easy to read and clear. Relationship lines are not crossed.

Are intersection tables properly formed (e.g. two FKs and facilitate a M:N relationship)?

Yes the intersection tables are properly formed with two FKs to facilitate the M:N relationship.

Does the sample data suggest any non-normalized issues, e.g. partial dependencies or transitive dependencies?

The sample data appears to satisfy 3NF, where each table has a primary key and there are no composite keys. There are no partial or transitive dependencies.

In the SQL, are the data types appropriate considering the description of the attribute in the database outline?

Yes the data types are appropriate.

In the SQL, are the primary and foreign keys correctly defined when compared to the Schema?

Are appropriate CASCADE operations declared?

Yes the primary and foreign keys are correctly defined compared to the schema. Yes there are appropriate cascade operations.

In the SQL, are relationship tables present when compared to the ERD/Schema?

Yes.

In the SQL, is all example data shown in the PDF INSERTED?

Yes.

Is the SQL well-structured and commented (e.g. hand authored) or not (e.g. exported from MySQL)?

Yes comments are clear to introduce the project and act as labels for the tables as well.

As a reviewer, clearly describe to what extent your feedback to the team was original (e.g. "all my work") or non-original (e.g. used AI tools per Code Citation Tips).

My review is original.

Review by TA Razeel Hassan Ibrahim Rasheed:

For your DDL,

- For your DDL, how will you handle anomalies created by deleting data? Your foreign keys use ON DELETE RESTRICT. Please consider whether deleting related records should be restricted, cascaded, or handled another way. See Exploration - MySQL Cascade: https://canvas.oregonstate.edu/courses/2042369/pages/exploration-mysql-cascade?module_item_id=26640179.
- Please do check your 1:M relationship descriptions in outline, because 1:M and M:1 are different, this would cause deduction in points if the error persists (Ignore if this irrelevant).
- Please make sure to include all verbatim feedback in the PDF submission as required in the project instructions, even if it increases the length of the document.
- In DB outline, define the M:N relationship in main entities not just in junction table.

Step 2 Accepted Fixes

- SQL comment syntax - one reviewer noted a SQL syntax error when importing the DDL via phpMyAdmin. This was caused by a "--comment" style with no space which does not conform to the SQL standard. All "--" comments in DDL.sql and DML.sql have been updated to "-- comment" with a space to conform to SQL syntax and to prevent future import errors.
- Attribute order of bikeYear and engineSize in the ERD and schema diagrams - one reviewer noted these attributes appear in a different order between the two diagrams. The ordering was changed in the ERD to match the schema diagram and database outline.
- Add M:N relationship to main entity outlines - The M:N relationship between Customers and Bikes has been added to the Relationships section of both the Customers and Bikes entities in the Database Outline. Previously, this relationship was only documented under the Rentals Intersection table. No changes to the ERD were made as the direct connector between Customers and Bikes was correctly removed in Step 1.

Step 2 Declined Suggestions

- bikeYear spacing inconsistency - one reviewer noted a space in "bike Year" in the ERD, however upon review there is no such spacing error present. The ERD correctly displays "bikeYear" consistent with the schema and database outline. No change was made.
- ON DELETE RESTRICT behavior - We were asked to consider whether deleting related records should be restricted, cascaded, or handled another way. ON DELETE RESTRICT is intentional for both FK relationships in Rentals. Deleting a customer or bike that has associated rental records would create orphaned history and data loss, which is unacceptable for an operational rental system. RESTRICT enforces that records

must be manually reviewed and resolved before any deletion can occur.

Note: This position was revised in Step 3 - see Step 3 Accepted Fixes

- 1:M and M:1 relationships direction - Feedback flagged a potential inconsistency between 1:M and M:1 descriptions. After review, the descriptions are correct as written. Each relationship is described from the perspective of the entity being outlined. Customers and Bikes describe their relationships as 1:M toward Rentals, while the Rentals entity describes its relationships as M:1 toward Customers and Bikes. No changes were made.

Step 3 Feedback Verbatim

Review by Jeremy Hout:

Does the UI utilize a SELECT for every table in the schema? In other words, data from each table in the schema should be displayed on the UI.

Yes, the data from the DDL file is correctly displayed in the UI.

Does the UI implement an INSERT form for at least one table in the schema? In other words, there should be UI input fields that correspond to at least one table.

Yes, the INSERT queries are present for all four tables, including correct population methods such as text field or dropdowns.

Does the UI have at least one DELETE for any one entity? In other words, is there a form/link/button that will allow the deletion of a row in at least one table?

Yes, DELETE queries are present for all four tables, each correctly targets a single record.

Does the UI have at least one DELETE that will remove things from a M:M relationship?

Yes, based on the use case of the business, the way their restrictions and cascades are set up makes sense based on the relationships between the Customer, the Bikes, and the Rentals. It looks like the only way to delete from the M:N relationship is by deleting the Rental record, which looks possible.

Is there at least one UPDATE form in the UI for any one entity? In other words, in the case of Products, can productName, listPrice, qtyOnHand, e.g. be updated for a single ProductID record?

Yes, UPDATE queries are present for all four tables and are well written.

Is there at least one UPDATE form in the UI to modify an M:M relationship? In other words, does the UPDATE allow the user to select a different foreign key value to update the intersection table with?

Yes, the UPDATE form is robust and covers the ability to update different FK values.

Do you have any other suggestions for the team to help with their HTML UI? For example, using AS aliases to replace obscure column names such as fname with First Name.

This is a very strong submission. I think the layout of the website is very well organized and user friendly. The DML and DDL files are very organized as well. I don't have much feedback for this submission, I probably will even take some cues on how clean the layout of the webpage is.

All content is written by me.

Review by Chewon Cho:

Hi Group 25, thank you for sharing your work! I liked your thorough implementation and your dedication on your project. The following are my answers to the questions provided in Step 3 Review.

- **Does the UI utilize a SELECT for every table in the schema? In other words, data from each table in the schema should be displayed on the UI. Note: it is generally not acceptable for**

just a single query to join all tables and displays them.

The UI utilizes a SELECT for all four tables (Customers, Brands, Bikes, and Rentals) in the schema. Data from each table in the schema is displayed on the UI.

- **Does the UI implement an INSERT form for at least one table in the schema? In other words, there should be UI input fields that correspond to at least one table.**

Yes, the UI has INSERT forms for Customers, Brands, Bikes, and Rentals tables in the schema. Each INSERT form has UI input fields corresponding to each table. The input fields correspond to all related attributes of that table.

- **Does the UI have at least one DELETE for any one entity? In other words, is there a form/link/button that will allow the deletion of a row in at least one table?**

Yes, the UI has DELETE buttons for all four tables in the schema. There is a delete button at the end of every row that will allow the deletion of each row from each table.

- **Does the UI have at least one DELETE that will remove things from a M:M relationship? In other words, if an order is deleted from the Orders table, it should also delete the corresponding rows from the OrderDetails table, BUT it should not delete any Products or Customers.**

The DELETE functionality in the UI is not fully implemented yet, but by considering the DELETE buttons in the UI for all four tables in the schema, the DDL.sql, and the DML.sql, it has at least one DELETE for the Rentals entity, which is the composite entity that displays the M:M relationship between the Customers entity and Bikes entity. However, although the UI has one DELETE that will directly remove a row from Rentals entity, the DELETE might not be able to automatically remove rows from the M:M relationship after implementation, since there is no such CASCADE implementation as ON DELETE CASCADE in the Rentals entity in the DDL.sql. Due to the absence of ON DELETE CASCADE in the Rentals entity referencing the Customers(customerID), if a customer is deleted from the Customers table, it does not also delete the corresponding rows from the Rentals table. Similarly, since there is no ON DELETE CASCADE in the Rentals entity referencing the Bikes(bikeID), if a bike is deleted from the Bikes table, it does not also delete the corresponding rows from the Rentals table.

Using the ON DELETE CASCADE for these parts might help the UI fully implement DELETES that will remove things from a M:M relationship.

- **Is there at least one UPDATE form in the UI for any one entity? In other words, in the case of Products, can productName, listPrice, qtyOnHand, e.g. be updated for a single ProductID record?**

Yes, the UI implements UPDATE forms for Customers, Brands, Bikes, and Rentals entities. There is an Edit button in the Action column of each row that will navigate the user to another page that shows the UPDATE form for that record. The UI input fields in the UPDATE form show the current data of that record, and the user can update any part of the field data.

In the UI, the form enables all attributes in each of those entities to be updated for a single record, except for the PK since each Edit is already linked to a specific PK selected by the user.

- **Is there at least one UPDATE form in the UI to modify an M:M relationship? In other words, does the UPDATE allow the user to select a different foreign key value to update the intersection table with?**

Yes, there is one UPDATE form in the UI to modify an M:M relationship between the Customers entity and Bikes entity. The Rentals entity is the composite entity which shows the M:M relationship between the Customers entity and Bikes entity. On the Rentals page, there is an Edit button in the Action column of each row that will navigate the user to another page that shows the UPDATE form for that record. In the UPDATE form, the form allows the user to select a different foreign key value to update the intersection table with. For example, when the user clicks the Edit

button on any row in the Rentals page, the user can select any Customer from the dropdown and any Bike from the dropdown, which can be different FK values than its original FK values.

- ***Do you have any other suggestions for the team to help with their HTML UI? For example, using AS aliases to replace obscure column names such as fname with First Name.***

The Rentals table on the Rentals page does not seem to be in ascending order of the PK (ID) values, while the tables of all the other pages are in ascending order of their PK values. Ordering the Rentals table following this order might help the UI to have further consistency in the design. Also, the styles of some buttons are inconsistent. There are multiple buttons in this UI. "Add Customer" and "Update Customer" buttons are black boxes with white text, while "Cancel" and "Edit" buttons are just blue texts without boxes. The "Cancel" button is also underlined, unlike the other buttons. The "Delete" button is a red box with white text. Although this might be intended, making the layouts of the buttons more consistent might save time for the users to find the buttons.

- ***As a reviewer, clearly describe to what extent your feedback to the team was original (e.g. "all my work") or non-original (e.g. used AI tools per Code Citation Tips).***
The feedback is all my work.

Review by Peter Ancheta:

- *Does the UI utilize a SELECT for every table in the schema?* In other words, data from each table in the schema should be displayed on the UI. Note: it is generally not acceptable for just a single query to join all tables and displays them.
 - Yes the UI utilizes select for every table.
- *Does the UI implement an INSERT form for at least one table in the schema?* In other words, there should be UI input fields that correspond to at least one table.
 - Yes the UI implements insert and each page has an "Add New" form.
- *Does the UI have at least one DELETE for any one entity?* In other words, is there a form/link/button that will allow the deletion of a row in at least one table?
 - Yes each table includes a delete button.
- *Does the UI have at least one DELETE that will remove things from a M:M relationship?* In other words, if an order is deleted from the Orders table, it should also delete the corresponding rows from the OrderDetails table, BUT it should not delete any Products or Customers.
 - Yes the Rentals page allows deleting a rental between a customer and bike.
- *Is there at least one UPDATE form in the UI for any one entity?* In other words, in the case of Products, can productName, listPrice, qtyOnHand, e.g. be updated for a single ProductID record?
 - Yes each table includes an edit button.
- *Is there at least one UPDATE form in the UI to modify an M:M relationship?* In other words, does the UPDATE allow the user to select a different foreign key value to update the intersection table with?
 - Yes the Rentals page has an edit option the allows updating of fields.
- *Do you have any other suggestions for the team to help with their HTML UI? For example, using AS aliases to replace obscure column names such as fname with First Name.*
 - Excellent work here Group 25! No notes on my end. The UI is clean and straightforward to use.
- *As a reviewer, clearly describe to what extent your feedback to the team was original (e.g. "all my work") or non-original (e.g. used AI tools per Code Citation Tips).*
 - This is my work.

No TA feedback or fourth peer review for Step 3 Draft. No TA feedback for Step 3 Final.

Step 3 Accepted Fixes

- Rentals page row ordering - one reviewer noted that the Rentals browse page did not display rows in ascending order of the rentalID primary key, while the other entity pages did. ORDER BY rentalID ASC was added to the Rentals browse query to fix this issue.
- Rentals ON DELETE behavior on foreign keys - one reviewer suggested implementing ON DELETE CASCADE on both Rentals FKs to support cascading deletion through the M:N relationship between Customers and Bikes. In Step 2 we declined a similar suggestion on the grounds that rental records should be preserved for ongoing maintenance decisions regardless of which customer was on the bike. Upon further review, we recognized that ON DELETE RESTRICT created a limitation in the design. The system claimed to support customer deletion, but in practice any customer with rental records could never be removed without first manually deleting their rental history. This left the design unable to fully demonstrate the M:N deletion behavior described in the project guidelines. To resolve this, both the customerID and bikeID foreign keys in Rentals have been updated to ON DELETE CASCADE. This change satisfies the assignment's requirement that parent entity deletion propagate cleanly through the intersection table without creating data anomalies.
- Button styling consistency - one reviewer noted that the action elements across the UI had inconsistent styling. Add, Update, and Delete were styled as buttons, but Edit and Cancel elements were rendered as plain text links. To resolve this, Edit and Cancel anchor tags were styled to match the button format. Navigation links on the index page were intentionally left as text links since they represent navigation rather than action.

Step 3 Declined Suggestions

- No suggestions were declined in this round of feedback.

Citations

The web application scaffolding and UI polish for Step 3, as well as the stored procedures and corresponding post routes for Step 4 were written with the help of Claude (Anthropic). Prompts described our database schema and requested UI scaffolding templates for a Node.js/Express/Handlebars application, minor styling updates, and SQL/routing patterns consistent with the course-provided starter code.

All other work in this document was done without the use of AI tools.